НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОСТРОЗЬКА АКАДЕМІЯ»
Кафедра інформаційних технологій та аналітики даних

Лабораторна робота №1. Налаштування середовища розробки для HTML, CSS, JavaScript/TypeScript

з дисципліни «Сучасна веброзробка: HTML, CSS, JS/TS»

> **Примітка щодо актуальності.** Документ підготовлено станом на 10 вересня 2026 року. Назви меню, версії IDE, плагінів та AI-функцій можуть змінюватися; у разі відмінностей слід орієнтуватися на актуальну офіційну документацію відповідного продукту.

1. Мета роботи

Сформувати працездатне локальне середовище фронтенд-розробки для виконання наступних лабораторних робіт з HTML, CSS, JavaScript/TypeScript: редактор коду або IDE, інструменти якості коду, локальний перегляд вебсторінок, Git/GitHub, публікація через GitHub Pages і Vercel, а також кероване використання AI-агента.

Очікуваний результат: студент може самостійно відкрити проєкт, внести зміну, перевірити її локально, зберегти зміни в Git, оновити GitHub-репозиторій і переконатися, що зміна опублікована на GitHub Pages та Vercel.

2. Цілі роботи

- дослідити роль HTML, CSS, JavaScript та TypeScript у сучасному фронтенд-проєкті та зрозуміти, де шукати нормативну й практичну документацію;
- порівняти VS Code, WebStorm, Cursor та Windsurf за критеріями підтримки вебтехнологій, розширюваності, Git, налагодження та AI-можливостей;
- встановити мінімальний набір інструментів для форматування, статичного аналізу та локального перегляду;
- налаштувати AI-агента через правила/інструкції, повторювані процедури та якісні промпти;
- створити або перевірити GitHub-акаунт, репозиторій і GitHub Pages;
- створити або перевірити Vercel-акаунт і виконати перший деплой з GitHub;
- сформувати звичку перевіряти AI-зміни, результати линтера, diff та фактичну поведінку сторінки перед commit/push.

3. Теоретична частина

3.1. HTML, CSS, JavaScript та TypeScript: що саме ми налаштовуємо

HTML описує структуру документа й семантику контенту; CSS відповідає за представлення та компонування; JavaScript додає поведінку й програмну логіку в браузері; TypeScript розширює JavaScript статичною типізацією та компілюється у JavaScript. Для навчання важливо розрізняти мовні стандарти та інструменти, які допомагають їх писати, перевіряти й запускати.

Офіційний HTML Living Standard є живою специфікацією WHATWG і має окрему «Edition for Web Developers». Офіційна екосистема CSS описується через W3C, зокрема через CSS Snapshot 2026, який узагальнює поточний стан CSS-модулів. JavaScript формально стандартизується Ecma International у ECMA-262; для

практичної роботи з мовою найзручнішим довідником є MDN. TypeScript має власний Handbook і Reference, з яких Handbook позиціонується як основна щоденна навчальна документація.

Технологія	Основне офіційне джерело	Що дає студенту	Чи описує налаштування IDE?
HTML	WHATWG HTML Living Standard	семантика, структура документа, DOM, алгоритми браузера	Ні, переважно специфікація; setup — у документації інструментів
CSS	W3C CSS / CSS Snapshot 2026	модулі CSS, синтаксис, каскад, layout, сумісність	Мінімально; інструменти авторингу згадуються загально
JavaScript	ECMAScript + MDN	мовні конструкції, API та практичні приклади	MDN має окремий Environment Setup, але це не документація конкретної IDE
TypeScript	TypeScript Handbook / Reference	типізація, compiler options, everyday types, modules	Пояснює tooling, але IDE setup деталізується в документації IDE

3.2. Де офіційна документація говорить про IDE та environment setup

Аналіз показує корисний поділ на три рівні. Перший рівень — стандарти: вони визначають мову/платформу, а не конкретний редактор. Другий — навчальні ресурси MDN і TypeScript, де вже з'являються рекомендації щодо редактора, терміналу, браузера та локального середовища. Третій — документація конкретної IDE, де описано settings, plugins/extensions, linting, debugging, previews, workspace configuration та AI.

- MDN має окремий Environment Setup module із матеріалами про текстовий редактор, браузери, файлову систему і командний рядок; це найпряміше офіційне джерело для первинного навчального setup.
- VS Code має окремі офіційні сторінки для HTML, CSS/SCSS/Less та TypeScript, включно з IntelliSense, форматуванням, дебагом і роботою з розширеннями.
- WebStorm документує глобальні та project-level settings, плагіни, підтримку HTML/CSS/JS/TS і інтеграцію ESLint/Prettier.
- Cursor і Windsurf позиціонуються як AI-first середовища, тому їх документація додає Agent/Cascade, rules, skills, context та інші AI-компоненти поверх знайомої IDE-моделі.

3.3. Порівняння IDE/редакторів

Критерій	VS Code	WebStorm	Cursor	Windsurf
Позиціонування	легкий розширюваний редактор з IDE-подібними функціями	повноцінна IDE JetBrains для JavaScript/TypeScript/web	VS Code-based AI-first editor	VS Code-based AI-first editor
HTML/CSS/JS/TS	сильна built-in підтримка	глибока built-in підтримка	як у VS Code + AI	як у VS Code + AI
Розширення	Visual Studio Marketplace	JetBrains Marketplace + bundled plugins	Open VSX + власні/ аудитовані заміни для частини розширень	Open VSX / сумісні плагіни, залежно від версії
Git	built-in + extensions	глибока built-in інтеграція	built-in	built-in
Debugging	JavaScript/TypeScript/Node.js	клієнтський і серверний JS, інтеграція тестів	через VS Code stack + Agent	через VS Code stack + Cascade
AI	GitHub Copilot / інші плагіни	Junie / AI Assistant та інші плагіни	Agent, rules, skills, MCP та ін.	Cascade, rules/memories/workflows/skills за актуальною документацією
Рекомендація для курсу	основний baseline	сильна альтернатива	AI-first альтернатива	AI-first альтернатива

3.4. Форматування, linting та локальний запуск

Prettier і ESLint вирішують різні задачі. Prettier нормалізує представлення коду; ESLint аналізує код за правилами якості, стилю та потенційних помилок. Для навчання корисно бачити обидва класи інструментів: форматування не означає, що код правильний, а linting не повинен перетворюватися на ручне форматування кожного пробілу.

Для статичного HTML/CSS/JS-проєкту потрібен локальний server/preview, щоб перевіряти поведінку в браузері в умовах, близьких до реального web environment. Microsoft Live Preview дає локальний сервер і live reload; WebStorm має власний preview та web tooling.

3.5. Git, GitHub, GitHub Pages і Vercel

Git зберігає локальну історію змін; GitHub — віддалений репозиторій і спільний простір для коду. GitHub Pages публікує статичний сайт без окремого хостингу, а Vercel може бути підключений до Git-репозиторію й автоматично запускати deployment після push. Це дозволяє в межах однієї лабораторної пройти повний цикл: edit → verify → commit → push → deployment.

- **GitHub Pages:** для базового варіанту достатньо репозиторію з entry file `index.html` і налаштування Pages на публікацію з branch/директорії.
- **Vercel:** після імпорту репозиторію створюється проект; push до підключеного Git-репозиторію запускає новий deployment, а PR може мати preview deployment.

3.6. AI-агент: чат, агент, правила, skills

У сучасних IDE AI-функція — це вже не лише автодоповнення. AI-агент може шукати файли, аналізувати репозиторій, редагувати кілька файлів, запускати команди та тести. Тому студентові потрібно навчитися не тільки писати промпт, а й визначати межі автономії агента, перевіряти його diff та відокремлювати «контекст про проєкт» від «команди для конкретного кроку».

Механізм	Призначення	Приклад	Коли використовувати
Prompt	разове завдання	«Знайди причину 404 і виправ її»	одна конкретна задача
Rule / AGENTS.md	постійні правила проєкту	«Використовуй TypeScript; перед commit запускати lint»	спільні або постійні вимоги
Skill (SKILL.md)	повторювана процедура + знання	«Перевір frontend за чеклістом доступності»	складний, але типовий workflow
Workflow / command	ручний запуск послідовності	«/deploy-preview»	коли процедура має виконуватися явно
MCP / tools	доступ до зовнішнього контексту або сервісу	документація, issue tracker	коли потрібні зовнішні дані/дії

Cursor офіційно підтримує Agent Skills як відкритий стандарт, правила проєкту та `AGENTS.md`. Опис Agent Skills визначає переносимий підхід: папка зі `SKILL.md`, яка може містити інструкції, scripts, references та assets. GitHub Copilot підтримує repository-wide instructions у `.github/copilot-instructions.md` та `AGENTS.md` для agent instructions; Junie також використовує `AGENTS.md`. Для Windsurf актуальна документація описує агентні правила, пам'ять і повторювані workflows/skills; конкретні назви меню можуть змінюватися.

3.7. Як писати промпти

Ефективний промпт для coding agent повинен містити не «будь ласка, зроби красиво», а контекст, бажаний результат, обмеження та критерії перевірки. Особливо корисно розрізняти intent context (що хочемо отримати) і state context (що зараз є в проєкті): файл, помилка, stack trace, screenshots, вимоги до браузера тощо.

- Спочатку просити агента коротко описати план і файли, які він збирається змінити.
- Для невеликої лабораторної давати одну функціональну задачу на одну ітерацію.
- Вказувати критерії готовності: lint без помилок, сторінка відкривається, кнопка працює, mobile viewport перевірений.

- Просити не змінювати конфігурацію/залежності без пояснення причини.
- Після зміни обов'язково перевіряти diff, запускати локальну перевірку і лише потім commit/push.

3.8. Академічна доброчесність та безпечне використання AI

AI-агент є інструментом підтримки розробника, а не джерелом істини. Його пропозиції можуть бути неповними, застарілими або непридатними до конкретного проєкту. Не слід передавати в AI секрети, токени, паролі, приватні ключі чи персональні дані, якщо це не передбачено політикою та налаштуваннями інструмента. Студент повинен уміти пояснити зміни, які прийняв у свій код, і продемонструвати, що перевірив їх фактично.

4. Завдання лабораторної роботи

1. Проаналізувати документацію HTML/CSS/JavaScript/TypeScript щодо середовища розробки та визначити, які рекомендації належать до стандартів, а які — до IDE/tooling.
2. Встановити та налаштувати одну з рекомендованих IDE: VS Code, WebStorm, Cursor або Windsurf.
3. Перевірити наявність підтримки HTML, CSS, JavaScript, TypeScript, Git і браузерного debugging/preview.
4. Встановити сторонні інструменти відповідно до обраної IDE (мінімум ESLint + Prettier; для редактора без вбудованого preview — Live Preview або аналог).
5. Створити короткий статичний web-проєкт з файлами `index.html`, `styles.css`, `script.js` або `script.ts` та `README.md`.
6. Налаштувати AI-агента: створити `AGENTS.md` або еквівалентні project rules; за можливості створити один Skill із `SKILL.md`.
7. Створити GitHub-репозиторій, виконати перший commit і push.
8. Увімкнути GitHub Pages і отримати публічне посилання.
9. Підключити репозиторій до Vercel та отримати deployment URL.
10. Внести мінімальну зміну в сторінку, перевірити її локально, виконати commit/push і переконатися, що GitHub Pages та Vercel оновилися.
11. Підготувати звіт за вимогами розділу 6.

5. Хід виконання роботи

5.1. Підготовка робочого середовища

1. Встановити Git та Node.js LTS. На момент підготовки документа актуальна LTS-гілка Node.js — 24.x; конкретний patch-реліз слід брати з офіційної сторінки завантаження.

```
git --version
node --version
npm --version
```

2. Встановити сучасний браузер з Developer Tools. Рекомендується мати Chrome або Edge, а також другий браузер для базової перевірки.
3. Створити GitHub-акаунт та увійти в нього. Для лабораторної репозиторій має бути доступний викладачу за наданим посиланням.
4. Створити Vercel-акаунт і за можливості прив'язати його до GitHub.

5.2. Налаштування VS Code (рекомендований baseline)

1. Встановити Visual Studio Code з офіційного сайту.
2. Відкрити кореневу папку майбутнього проєкту через **File** → **Open Folder**.
3. Переконатися, що HTML, CSS та JavaScript/TypeScript визначаються як відповідні мови й працюють IntelliSense/Emmet.

4. Встановити розширення: ESLint, Prettier - Code formatter, Live Preview. GitLens та Error Lens — за бажанням.
5. Створити файл `.vscode/extensions.json` із рекомендаціями для workspace.

```
{
  "recommendations": [
    "dbaeumer.vscode-eslint",
    "esbenp.prettier-vscode",
    "ms-vscode.live-server"
  ]
}
```

6. Створити `.vscode/settings.json` та зафіксувати базові параметри.

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit"
  },
  "editor.tabSize": 2,
  "editor.insertSpaces": true,
  "files.trimTrailingWhitespace": true,
  "files.insertFinalNewline": true
}
```

7. Створити `.prettierrc.json`, щоб форматування було переносимим між IDE.

```
{
  "printWidth": 100,
  "tabWidth": 2,
  "useTabs": false,
  "semi": true,
  "singleQuote": true
}
```

8. Для JavaScript/TypeScript створити ESLint-конфігурацію відповідно до актуальної версії ESLint. Для ESLint 9 рекомендований формат — `eslint.config.js` / `.mjs` / `.cjs` / `.ts` / `.mts`.

5.3. Налаштування WebStorm

1. Встановити WebStorm та відкрити кореневу папку проєкту.
2. Перевірити **Settings** → **Plugins**, а також Languages & Frameworks / Editor налаштування.
3. Перевірити, що HTML, CSS, JavaScript і TypeScript підтримуються без додаткових мовних пакетів.
4. Увімкнути/configure ESLint у проєкті. WebStorm показує lint warnings/errors безпосередньо в редакторі.
5. За потреби встановити/налаштувати Prettier через JetBrains Marketplace або через `package.json` проєкту.
6. Налаштувати форматування на основі єдиного `.prettierrc.json`, щоб не мати різних правил форматування в різних IDE.

5.4. Налаштування Cursor

1. Встановити Cursor, увійти в акаунт і відкрити проєкт.
2. Перевірити імпорт VS Code settings/extensions, якщо ви переходите з VS Code.
3. Переконатися, що AI Agent бачить корінь репозиторію і може знаходити HTML/CSS/JS/TS файли.
4. Створити `AGENTS.md` у корені проєкту для загальних правил.
5. Створити `.agents/skills/frontend-review/SKILL.md` або `.cursor/skills/frontend-review/SKILL.md`. Рекомендовано `.agents/skills` як більш переносимий варіант для сумісних агентів.

5.5. Налаштування Windsurf

1. Встановити Windsurf, завершити onboarding та відкрити корінь проєкту.
2. Перевірити імпорт конфігурації з VS Code або Cursor, якщо це доречно.
3. Перевірити Extensions/Plugins та встановити лише ті frontend-інструменти, яких бракує конкретній збірці.
4. Налаштувати Cascade: правила/інструкції, повторювані workflows/skills та обмеження на зміну файлів.
5. Перевірити, що AI-агент може працювати з терміналом, але студент переглядає команди перед прийняттям змін, особливо команди, що видаляють файли або змінюють залежності.

5.6. Створення навчального web-проєкту

1. Створити структуру:

```
frontend-lab-1/
├── index.html
├── styles.css
├── script.js
├── README.md
├── .gitignore
├── .editorconfig
├── .prettierrc.json
├── .vscode/
│   ├── settings.json
│   └── extensions.json
├── .github/
│   └── copilot-instructions.md   # якщо використовується Copilot
└── AGENTS.md                     # загальні інструкції для AI
```

2. У `index.html` реалізувати просту сторінку: заголовок, абзац, кнопку, інформаційний блок та футер.
3. У `styles.css` зробити responsive layout для desktop і mobile viewport.
4. У `script.js` реалізувати мінімальну інтерактивну дію: зміна тексту, перемикання класу, theme switch або лічильник.
5. Додати `README.md` з назвою проєкту, коротким описом, способом локального запуску та посиланнями на GitHub Pages/Vercel після деплою.

5.7. Налаштування AI-агента

1. Створити `AGENTS.md`.

```
# Frontend Lab Agent Instructions

## Role
Act as a careful frontend development assistant for a 3rd-year CS student.

## Project
- HTML + CSS + JavaScript/TypeScript.
- Prefer browser-native APIs and simple solutions.
- Do not add dependencies unless there is a clear reason.

## Before editing
1. Inspect the relevant files.
2. State a short plan.
3. Identify risks or missing context.

## After editing
1. Check the diff.
2. Run the relevant lint/format/type checks.
3. Report exactly what changed and what was verified.

## Safety
Never expose or invent secrets. Do not delete files or rewrite project
configuration without explaining why.
```

2. Створити один Skill, наприклад `frontend-review`.

```
---
name: frontend-review
description: Review a small frontend change for correctness, accessibility, responsiveness,
and maintainability. Use when the student asks for a pre-commit review or before deployment.
---

# Frontend Review

## Checklist
1. Inspect only the files relevant to the task.
2. Check HTML semantics and obvious accessibility issues.
3. Check CSS responsiveness and class naming consistency.
4. Check JavaScript/TypeScript for runtime errors and unnecessary complexity.
5. Run the project's available lint/type checks.
6. Summarize findings by severity: blocker, warning, suggestion.
7. Do not make changes unless explicitly asked or the agent workflow permits fixes.
```

3. Перевірити, що агент справді застосовує правила. Виконати тестовий запит: «Знайди проблеми в `index.html` та `styles.css`, але нічого не змінюй; покажи лише перелік проблем із поясненням».
4. Виконати другий запит на реальну зміну, наприклад: «Додай перемикач темної теми. Спочатку покажи план, після внесення змін запусти перевірки та коротко опиши diff».

5.8. Робота з Git та GitHub

1. У корені проєкту виконати:

```
git init
git add .
git commit -m "chore: initialize frontend lab 1"
git branch -M main
```

2. На GitHub створити порожній репозиторій без автоматичного README, якщо локальний проєкт уже має власний README.

3. Підключити remote та відправити код:

```
git remote add origin https://github.com/<username>/<repository>.git
git push -u origin main
```

4. Перевірити репозиторій у браузері. Він повинен містити вихідні файли, конфігурацію IDE та AI-інструкції, але не містити `node_modules`, секретів, локальних кешів чи службових файлів ОС.

5.9. GitHub Pages

1. Відкрити GitHub → repository → **Settings** → **Pages**.
2. У **Build and deployment** вибрати **Source** → **Deploy from a branch**.
3. Вибрати branch `main` і folder `/(root)`, якщо `index.html` лежить у корені.
4. Дочекатися публікації та відкрити URL GitHub Pages. Зафіксувати його для звіту.
5. Якщо виникає 404, перш за все перевірити наявність `index.html` у верхньому рівні обраного publishing source.

5.10. Vercel

1. Увійти на Vercel і вибрати **New Project / Import Git Repository**.
2. Надати Vercel доступ до потрібного GitHub-репозиторію та вибрати його.
3. Для простого статичного проєкту залишити автоматично визначені налаштування, якщо вони коректні. Перевірити root directory та build settings перед Deploy.
4. Запустити Deploy і відкрити сформований deployment URL.
5. Після наступного `git push` перевірити, що Vercel створив новий deployment. За можливості зафіксувати deployment URL у README.

5.11. Контрольна зміна та фінальна перевірка

1. Змінити один UI-елемент, наприклад текст кнопки, колір, theme toggle або новий блок.
2. Запустити локальний preview та перевірити desktop + mobile viewport.
3. Виконати форматування та linting. За наявності TypeScript — перевірити type errors.
4. Подивитися `git diff` і переконатися, що до commit потрапили лише потрібні файли.
5. Створити commit:

```
git add .
git commit -m "feat: update landing page interaction"
git push
```

6. Перевірити GitHub repository, GitHub Pages та Vercel після оновлення.

6. Вимоги до звіту

Звіт має бути коротким, доказовим і відтворюваним. Його завдання — показати не те, що «IDE встановлено», а те, що середовище реально працює і студент може пройти повний цикл зміни та публікації.

6.1. Приклад листа

Тема листа: `[Веб] ЛР-1: Налаштування середовища розробки`

У листі:

- GitHub repository: `https://github.com/<username>/<repository>`
- GitHub Pages: `https://<username>.github.io/<repository>/`
- Vercel: `https://<project>.vercel.app/`
- Обрана IDE.
- Список установлених ключових додатків/плагінів із коротким призначенням.

- Результат перевірки Git, Node.js та npm (версії).
- Опис налаштованого AI-агента: який саме агент, де зберігаються правила.
- Короткий висновок до лабораторної роботи.

6.2. Приклад структури звіту

Розділ	Зміст
1. Тема і мета	назва роботи, мета, ПІБ, група
2. Середовище	IDE, ОС, Git, Node.js, основні плагіни
3. AI-агент	агент, AGENTS.md/rules, skill, 2–3 приклади промптів
4. GitHub	repository, commit/push, посилання
5. Deployment	GitHub Pages і Vercel, посилання
6. Контрольна зміна	що змінилось, як перевірялось, як оновились deployment-и
7. Висновок	що налаштовано, що перевірено, що було складним

7. Контрольні питання

1. Чим відрізняються HTML Living Standard, CSS specification, ECMAScript specification та TypeScript Handbook?
2. Чому стандарт мови не визначає конкретну IDE?
3. Яка різниця між formatter і linter?
4. Чому `.prettierrc` та IDE settings бажано доповнювати проєктною конфігурацією?
5. Які переваги й недоліки VS Code порівняно з WebStorm для студента?
6. У чому принципова різниця між звичайним чат-асистентом та coding agent?
7. Коли варто використати `AGENTS.md`/rules, а коли Skill?
8. Що має перевірити студент перед тим, як прийняти AI-зміни?
9. Що запускає новий deployment у Vercel при підключеному Git-репозиторії?
10. Чому наявність `index.html` важлива для базового GitHub Pages deployment?

8. Критерії готовності лабораторної

Критерій	Мінімальна вимога
IDE	встановлена, відкриває проєкт, працюють HTML/CSS/JS/TS features
Tooling	ESLint + Prettier налаштовані; є локальний preview/debug
AI	агент доступний; створено `AGENTS.md` або equivalent rules; створено 1 reusable skill за підтримки інструмента
GitHub	репозиторій створено, код запушено, є щонайменше 2 змістовні commits
GitHub Pages	публічний URL відкриває актуальну версію сайту
Vercel	deployment успішний; після push створюється нова версія
Зміна	студент продемонстрував edit → verify → commit → push → deploy
Звіт	містить посилання, короткий опис середовища, AI та висновок

9. Список джерел

Нижче наведено переважно офіційні джерела виробників та організацій-розробників стандартів. Вони мають пріоритет над сторонніми статтями й відео під час виконання лабораторної.

1. WHATWG. HTML: The Living Standard — Edition for Web Developers
2. W3C. Cascading Style Sheets home page
3. W3C. CSS Snapshot 2026

4. ECMA International / TC39. ECMAScript 2025 Language Specification
5. MDN Web Docs. Web developer guides
6. MDN Web Docs. Environment Setup
7. MDN Web Docs. JavaScript Guide
8. MDN Web Docs. CSS — Getting started
9. TypeScript. The TypeScript Handbook
10. Visual Studio Code. HTML in Visual Studio Code
11. Visual Studio Code. CSS, SCSS and Less
12. Visual Studio Code. TypeScript in Visual Studio Code
13. Visual Studio Code. Extension Marketplace
14. Visual Studio Marketplace. ESLint
15. Visual Studio Marketplace. Prettier - Code formatter
16. Microsoft / VS Code. Live Preview
17. JetBrains. Configure WebStorm settings
18. JetBrains. HTML in WebStorm
19. JetBrains. CSS in WebStorm
20. JetBrains. JavaScript in WebStorm
21. JetBrains. TypeScript in WebStorm
22. JetBrains. Plugins in WebStorm
23. Cursor. Cursor — Installation
24. Cursor. Agent overview
25. Cursor. Rules
26. Cursor. Agent Skills
27. Cursor. Working with Context
28. GitHub. Adding repository custom instructions for GitHub Copilot
29. GitHub. Creating a GitHub Pages site
30. GitHub. Configuring a publishing source for GitHub Pages
31. GitHub. Creating a repository for your project
32. Vercel. Deploying Git Repositories with Vercel
33. Vercel. Deploying to Vercel
34. Node.js. Download Node.js
35. npm. Downloading and installing Node.js and npm
36. Agent Skills. Agent Skills Overview
37. Agent Skills / GitHub. Specification and documentation for Agent Skills
38. Anthropic. Prompting best practices
39. Windsurf / Devin Docs. Plugins / Windsurf
40. Windsurf / Devin Docs. Workflows
41. Windsurf / Devin Docs. Cascade overview